HARI 1

Arsitektur Microservices dan Fondasi Service Go

Hari 1: monolith dan microservices, penentuan *service boundary*, Go *essentials*, serta account service dan transfer service

DISIAPKAN UNTUK

Peserta In-house Training

Endy Muhardin

PT ArtiVisi Intermedia · Oktober 2026

HARI 1

Agenda Hari 1

Waktu	Topik	Format
08:00–08:45	Peta *concern* microservices dan penyedianya (tim, ekosistem, atau platform); ringkasan monolith dan microservices	Presentasi
08:45–09:30	*Service decomposition*: kapabilitas bisnis, *bounded context*, dekomposisi studi kasus perbankan	Presentasi dan diskusi
09:30–10:00	Ekosistem Go dibandingkan Java, Node.js, dan PHP; alasan memilih Go	Presentasi
10:15–11:30	Go *essentials* untuk service: *module*, *struct*, *interface*, *error handling*, *goroutine* dan *channel*, `net/http`	Presentasi dan demo
11:30–12:00	Anatomi service Go (1): struktur *project*, *routing* dan perbandingan *router*, validasi, *12-factor*, konfigurasi	Demo
12:00–13:00	Istirahat makan siang	—
13:00–13:30	Anatomi service Go (2): *liveness* dan *readiness*, *graceful shutdown*, pemetaan *error*	Demo
13:30–15:00	**Lab 1**: account service dengan REST, PostgreSQL, dan *health check*	*Hands-on*
15:15–16:30	**Lab 2**: transfer service sebagai service kedua, komunikasi REST pertama	*Hands-on*
16:30–17:00	Ringkasan Hari 1 pada peta *concern*, tanya jawab	Diskusi

PETA TRAINING

Materi dan lab selama empat hari

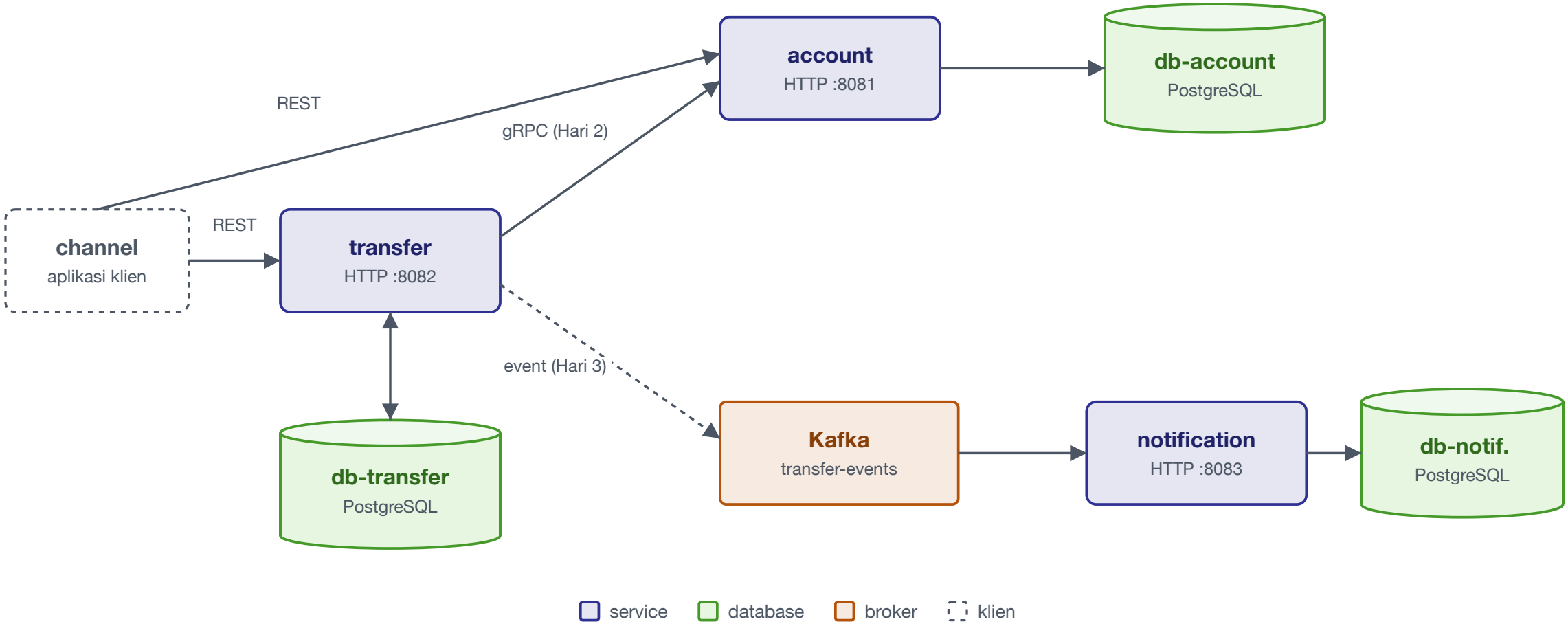
Hari	Tema	Lab
1	Arsitektur, Go *essentials*, fondasi service	Lab 1 account service; Lab 2 transfer service dengan REST
2	*Resiliency*, gRPC, dan desain saga	Lab 3 *timeout*, *retry*, *circuit breaker*; Lab 4 transfer ke account melalui gRPC
3	*Messaging*, konsistensi data, *security*, konsep *observability*	Lab 5 saga, *outbox*, Kafka; Lab 6 *idempotent consumer*; Lab 7 JWT
4	*Observability*, *deployment*, CI/CD	Lab 8 *tracing* di Grafana; Lab 9 *deploy* ke k3d; Lab 10 *pipeline* CI/CD

CATATAN

Setiap lab melanjutkan kode dari lab sebelumnya. Di akhir Hari 4, sistem yang sama sudah berjalan di Kubernetes dan dirilis melalui *pipeline* CI/CD.

STUDI KASUS

Sistem perbankan sederhana di akhir Hari 4



STUDI KASUS

Kondisi kode di akhir setiap lab

Lab	Yang ditambahkan ke sistem
1	Account service: REST, PostgreSQL, validasi, *health check*
2	Transfer service dengan database sendiri, memanggil account melalui REST
3	*Timeout*, *retry*, dan *circuit breaker* (failsafe-go) pada panggilan transfer ke account
4	Kontrak protobuf; transfer memanggil account melalui gRPC dengan *interceptor*
5	Saga dengan kompensasi, *transactional outbox*, Kafka, notification service
6	*Idempotent consumer* dan *idempotency key* pada `POST /transfers`
7	Token OAuth2 dari *authorization server*, validasi JWT di REST dan gRPC
8	*Structured log*, *correlation ID*, metrik Prometheus, *trace* OpenTelemetry, Grafana
9	Dockerfile *multi-stage* dan *manifest* Kubernetes, berjalan di k3d
10	*Pipeline* CI/CD per service: *test*, *build image*, *push*, *deploy*

Setiap kondisi akhir lab tersedia sebagai *git tag* (`lab-01` … `lab-10`). Peserta yang tertinggal melanjutkan dari *tag* lab sebelumnya dengan `git checkout lab-NN`.

PETA TRAINING

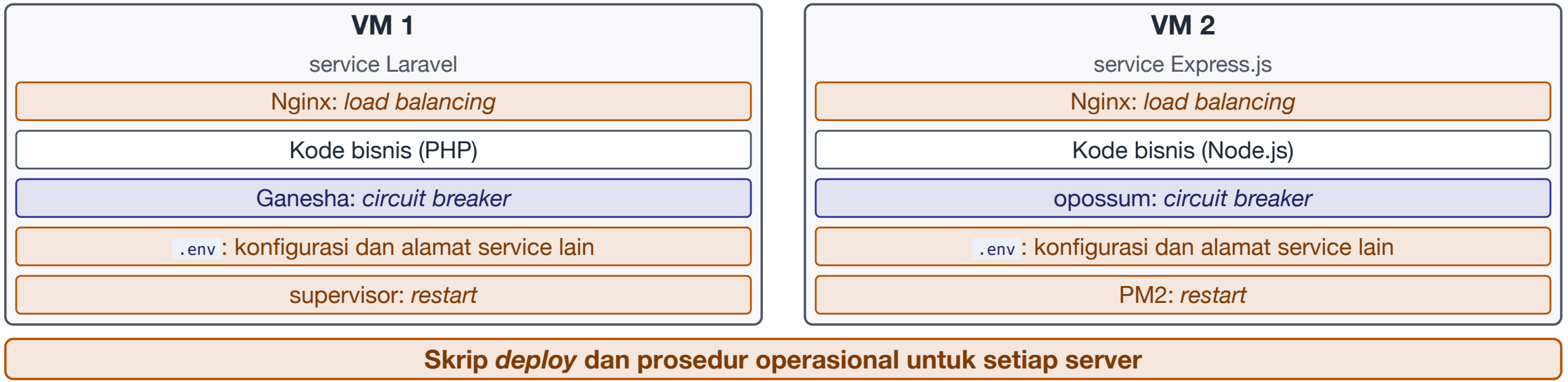
Peta *concern* microservices

#	*Concern*	Pertanyaan yang dijawab	Hari dan lab	Di kode (*library*)	Di platform atau infrastruktur
1	*Decomposition* dan kepemilikan data	*Service boundary* dan pemilik data	H1, Lab 1–2	—	*Database per service*
2	Komunikasi sinkron dan asinkron	REST, gRPC, *messaging*	H1–H3, Lab 2, 4, 5	`net/http`, grpc-go, kafka-go	Kafka
3	*Resiliency*	Tetap berfungsi saat dependensi lambat atau mati	H2, Lab 3–4	failsafe-go	*Probe*, *restart*, *rolling update* (H4)
4	Konsistensi data	Perubahan data lintas database	H2–H3, Lab 5–6	Saga, *outbox*, *idempotency*	—
5	*Security*	Identitas pengguna (SSO) dan service, serta hak akses	H3, Lab 7	JWT *middleware* dan *interceptor*	*Authorization server* atau IdP (Hydra di lab, Keycloak), API *gateway*
6	*Observability*	Kondisi tiap service dan integrasi antarservice	H3–H4, Lab 8	`slog`, Prometheus *client*, OTel SDK	Grafana, Loki, Tempo, Prometheus
7	*Configuration* (termasuk *centralized config* dan *secret*)	Nilai yang berbeda per *environment*	H1, Lab 1 (*env var*); H4, Lab 9 (ConfigMap, Secret)	`LoadConfig`	ConfigMap, Secret, *secret manager*
8	*Service discovery* dan *load balancing*	Alamat service tujuan	H4, Lab 9	— (alamat dari konfigurasi)	Kubernetes Service (DNS), Ingress
9	*Packaging* dan *deployment*	Cara service dijalankan dan diperbarui	H4, Lab 9	Dockerfile	Kubernetes Deployment, *probe*, *rolling update*
10	*Automation* (CI/CD)	Dari *commit* ke *production*	H4, Lab 10	Target `Makefile`	*Pipeline* (git server dan runner CI, Jenkins)

H1–H4 = Hari 1–4. Setiap hari dibuka dengan peta ini: baris yang disorot adalah *concern* yang dibahas hari itu.

PENYEDIA CONCERN

Tanpa platform: service *standalone* di VM



Setiap *concern* di luar kode bisnis disusun sendiri oleh tim, untuk setiap server

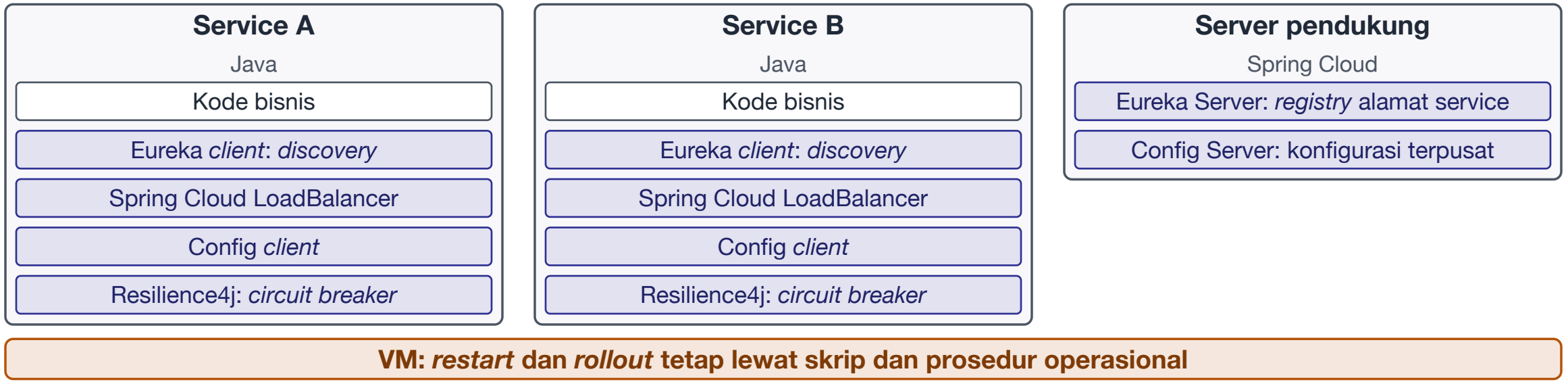
Alamat service lain ditulis tetap di `.env`. Menambah atau memindahkan server berarti mengubah konfigurasi di banyak tempat

Cara *restart* dan *rollout* berbeda per bahasa: PM2 untuk Node.js, supervisor untuk PHP

Ditulis dan dioperasikan tim | *Library* atau server ekosistem *tech stack* | Kode service

PENYEDIA CONCERN

Dukungan ekosistem: platform Spring Cloud



Spring Cloud menyediakan *discovery*, *load balancing*, konfigurasi, dan *circuit breaker* sebagai *library* beserta server pendukungnya

Dukungan ini hanya bisa digunakan oleh service JVM. Service dalam bahasa lain harus menulis sendiri *client* untuk Eureka dan Config Server

Restart dan *rollout* tidak termasuk, sehingga tetap menjadi pekerjaan tim

Ditulis dan dioperasikan tim | *Library* atau server ekosistem *tech stack* | Kode service

PENYEDIA CONCERN

Dukungan platform: Kubernetes



Platform menyediakan *discovery*, *load balancing*, konfigurasi, *restart*, dan *rollout* untuk service dalam bahasa apa pun

Kode service hanya memuat kebijakan yang memerlukan pengetahuan bisnis: *resiliency* per panggilan, konsistensi data, pemeriksaan *token*, dan instrumentasi

Studi kasus menggunakan model ini. Lab 9 dan Lab 10 menjalankannya di k3d dan di server pelatihan

Disediakan platform | *Library* atau server ekosistem *tech stack* | Kode service

PENYEDIA CONCERN

Service makin tipis ketika platform menangani lebih banyak *concern*

Standalone di VM

Satu service

- Kode bisnis
- *Library circuit breaker*

Disusun tim untuk setiap server

- Nginx
- `.env`
- PM2 atau supervisor
- Skrip *deploy*

Platform Spring Cloud

Satu service

- Kode bisnis
- Eureka *client*
- LoadBalancer
- Config *client*
- Resilience4j

Server ekosistem

- Eureka Server
- Config Server

Disusun tim

- *Restart* dan *rollout*

Platform Kubernetes

Satu service

- Kode bisnis
- *Resiliency*, *token*, instrumentasi

Disediakan platform

- *Discovery*
- *Load balancing*
- Konfigurasi
- *Restart*
- *Rollout*

Pada platform Kubernetes, *concern* yang sama untuk semua service ditangani platform. Kode service hanya memuat hal yang memerlukan pengetahuan bisnis, termasuk *resiliency* per panggilan.

Ditulis dan dioperasikan tim | *Library* atau server ekosistem *tech stack* | Disediakan platform | Kode service

PENYEDIA CONCERN

Ringkasan *concern* yang ditangani tim, ekosistem, atau platform

Concern	*Standalone* di VM (misalnya beberapa service Laravel atau Express.js)	Platform Spring Cloud	Platform Kubernetes
Service discovery	Alamat tetap di `.env` atau DNS internal	Eureka: *registry server* dan *client* di setiap service	Kubernetes Service dan DNS
Load balancing	Nginx atau HAProxy di depan setiap service	Spring Cloud LoadBalancer di sisi pemanggil	Kubernetes Service (kube-proxy)
Configuration	`.env` per server, disalin oleh skrip *deploy*	Spring Cloud Config Server dan *client*-nya	ConfigMap dan Secret
Restart dan *rollout*	PM2 atau supervisor, skrip *deploy* per server	Di luar *framework*: skrip dan prosedur operasional	Deployment: *probe*, *restart* otomatis, *rolling update*
Circuit breaker	*Library* per bahasa: opossum (Node.js), Ganesha (PHP)	Resilience4j	Tetap di kode: failsafe-go (Go), Resilience4j (Java)
Konsistensi, pemeriksaan token, instrumentasi	*Library* di setiap service	*Library* di setiap service	Tetap di kode
Disediakan oleh	Tim sendiri, untuk setiap service	Ekosistem Java, hanya untuk service JVM	Platform, untuk service dalam bahasa apa pun

CATATAN

Platform Spring Cloud adalah dukungan ekosistem satu *tech stack*. Platform Kubernetes adalah dukungan platform yang bisa digunakan untuk semua bahasa. Studi kasus menggunakan platform Kubernetes, sehingga kode Go hanya memuat kebijakan *resiliency*, konsistensi data, pemeriksaan token, dan instrumentasi.

PETA CONCERN MICROSERVICES

Concern yang dibahas di Hari 1

#	*Concern*	Cakupan di Hari 1
1	***Decomposition* dan kepemilikan data**	Lab 1–2: *service boundary* dari kapabilitas bisnis, satu database per service
2	**Komunikasi sinkron dan asinkron**	Lab 2: panggilan REST pertama dari transfer ke account
3	*Resiliency*	Lab 2: celah *client* tanpa *timeout* dicatat, ditangani Hari 2
4	Konsistensi data	Lab 2: celah credit gagal setelah debit dicatat, ditangani Hari 2–3
5	*Security*	Hari 3
6	*Observability*	Hari 3–4
7	***Configuration* (termasuk *centralized config* dan *secret*)**	Lab 1: konfigurasi wajib dari *environment variable* (faktor III)
8	*Service discovery* dan *load balancing*	Hari 4
9	***Packaging* dan *deployment***	Konsep *liveness*, *readiness*, dan *graceful shutdown*; YAML di Hari 4
10	*Automation* (CI/CD)	Target `Makefile` per service sudah disiapkan; *pipeline* di Hari 4

01

Monolith dan Microservices

Trade-off kedua arsitektur dan gejala *distributed monolith*

KONSEP

Ciri monolith

Ciri

- Seluruh fitur dirilis sebagai satu *deployable unit*
- Seluruh data tersimpan di satu database
- Modul saling memanggil melalui *function call* di dalam proses yang sama
- Satu transaksi ACID dapat mencakup perubahan di banyak tabel

Kelebihan yang sering terlupakan

- *Debugging* cukup di satu proses, dengan satu *stack trace*
- *Refactoring* lintas modul dibantu *compiler*
- Satu *pipeline* dan satu jadwal rilis
- Pemanggilan antarmodul tanpa latensi jaringan

CATATAN

Untuk banyak sistem, monolith modular dengan batas modul yang rapi tetap menjadi pilihan yang wajar.

KONSEP

Ciri microservices

Ciri

- Setiap service dirilis secara terpisah
- Setiap service memiliki database sendiri
- Service berkomunikasi melalui jaringan: REST, gRPC, atau *message broker*
- Tim pemilik service dapat merilis tanpa menunggu tim lain

Konsekuensi

- Panggilan jaringan bisa lambat atau gagal di tengah jalan
- Perubahan di dua service tidak dapat dibungkus satu transaksi database
- Jumlah *pipeline*, database, dan proses yang harus dipantau bertambah
- Penelusuran masalah membutuhkan *log* dan *trace* lintas service

KONSEP

Trade-off monolith dan microservices

Aspek	Monolith	Microservices
Deployment	Seluruh aplikasi dirilis sekaligus	Setiap service dirilis terpisah
Skalabilitas	Seluruh aplikasi diperbanyak bersama	Hanya service yang sibuk yang diperbanyak
Konsistensi data	Satu transaksi database	Saga, *outbox*, dan *idempotency* (Hari 2–3)
Kegagalan	Bila proses mati, seluruh aplikasi ikut berhenti	Kegagalan parsial, perlu *timeout* dan *circuit breaker* (Hari 2)
Debugging	*Stack trace* dalam satu proses	*Distributed tracing* lintas service (Hari 3–4)
Beban operasional	Satu aplikasi, satu database	Banyak *deployable unit*, *pipeline*, dan database

KONSEP

Kondisi yang membuat microservices merugikan

- **Tim kecil.** Bila semua service dikelola satu tim, pemecahan service tidak mengurangi koordinasi antartim, sedangkan beban operasionalnya tetap bertambah.
- ***Domain boundary* belum jelas.** *Service boundary* yang salah jauh lebih sulit diperbaiki dibandingkan *module boundary* di dalam satu aplikasi.
- **Belum ada otomasi.** Tanpa *pipeline* dan *observability*, setiap service tambahan menambah pekerjaan manual.
- **Beban merata.** Bila semua fitur menerima beban yang serupa, pemecahan service tidak meningkatkan skalabilitas.

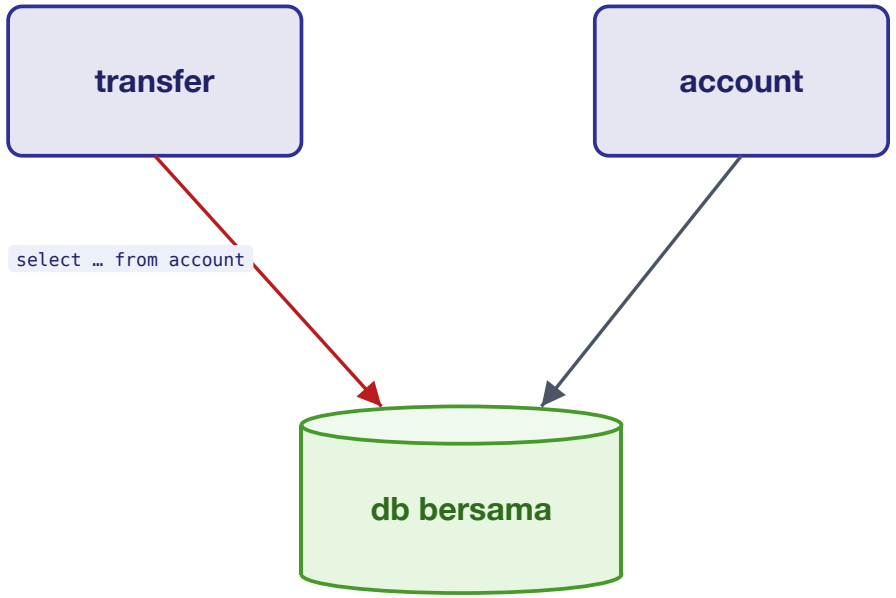
> **TIPS**
>
> Microservices layak dipertimbangkan ketika ada kebutuhan konkret: tim yang harus merilis secara independen, atau bagian sistem dengan kebutuhan skala yang berbeda jauh.

GEJALA

Gejala *distributed monolith*

Sistem sudah terpecah menjadi banyak service, tetapi masih harus dirilis dan dijalankan bersama-sama.

- Beberapa service harus di-*deploy* bersamaan
- Bila satu service mati, seluruh alur bisnis ikut berhenti
- Service membaca tabel milik service lain secara langsung
- Perubahan satu kontrak API memaksa banyak service ikut berubah



Transfer service membaca tabel `account` secara langsung, sehingga setiap perubahan skema tabel tersebut ikut merusak transfer service.

02

Service Decomposition

Service boundary menurut kapabilitas bisnis dan *bounded context*

KONSEP

Kapabilitas bisnis dan *bounded context*

Kapabilitas bisnis

Hal yang dikerjakan organisasi, terlepas dari cara aplikasi mengerjakannya: mengelola rekening, memindahkan dana, memberi tahu nasabah.

Kapabilitas bisnis jarang berubah, sehingga menjadi acuan *service boundary* yang stabil.

Bounded context

Batas berlakunya model domain. Di dalam batas tersebut, setiap istilah hanya memiliki satu arti.

Contoh: bagi account service, transfer hanyalah dua mutasi saldo. Bagi transfer service, transfer memiliki status, alasan gagal, dan riwayat.

KONSEP

Tiga acuan penentuan *service boundary*

1. **Service memiliki datanya sendiri.** Service lain hanya boleh mengakses data tersebut melalui API, tanpa *query* langsung ke tabelnya.
2. **Service dapat dirilis tanpa koordinasi lintas tim.** Rilis account service tidak menunggu jadwal rilis transfer service.
3. **Perubahan satu kebutuhan bisnis idealnya cukup di satu service.** Bila setiap fitur baru menyentuh tiga service, *service boundary* tersebut perlu ditinjau ulang.

> **CATATAN**
>
> Ketiga acuan ini sekaligus menjadi kriteria uji: *service boundary* yang melanggar salah satunya akan mengarah ke *distributed monolith*.

KESALAHAN UMUM

Pemecahan service yang keliru

Dipecah menurut *layer* teknis

Contoh: service “database”, service “validasi”, service “API”.

Setiap fitur baru menyentuh semua service tersebut, sehingga rilisnya tetap harus serempak.

Dipecah terlalu kecil

Contoh: satu service per tabel, misalnya service “nasabah”, service “saldo”, service “mutasi”.

Satu operasi bisnis membutuhkan banyak panggilan jaringan, dan konsistensi data harus dijaga lintas service untuk hal yang seharusnya cukup satu transaksi.

STUDI KASUS

Dekomposisi sistem perbankan

Service	Kapabilitas	Data yang dimiliki	Pemanggil
account	Mengelola rekening dan saldo	Tabel `account` dan `mutation`	Aplikasi klien (REST), transfer service
transfer	Memindahkan dana antarrekening	Tabel `transfer`	Aplikasi klien (REST)
notification	Memberi tahu hasil transfer (Hari 3)	Tabel `notification`	Tidak dipanggil langsung, tetapi membaca *event* dari Kafka

CATATAN

Transfer dipisahkan dari account karena aturan transfer (status, kompensasi, *idempotency*) dan aturan pengelolaan rekening berubah dengan alasan dan jadwal yang berbeda. Walaupun transfer menjadi service sendiri, saldo rekening hanya boleh diubah oleh account service.

KEPUTUSAN DESAIN

Database per service

Kebutuhan

Setiap service dapat mengubah skema dan dirilis tanpa koordinasi dengan pemilik service lain (acuan pertama dan kedua penentuan *service boundary*).

Opsi yang dipertimbangkan

- Satu database bersama untuk semua service
- Satu server database, dengan *schema* dan kredensial terpisah per service
- Satu server database per service

Pilihan di studi kasus

Setiap service memiliki *container* PostgreSQL sendiri. Service lain hanya mengakses datanya melalui API atau *event*.

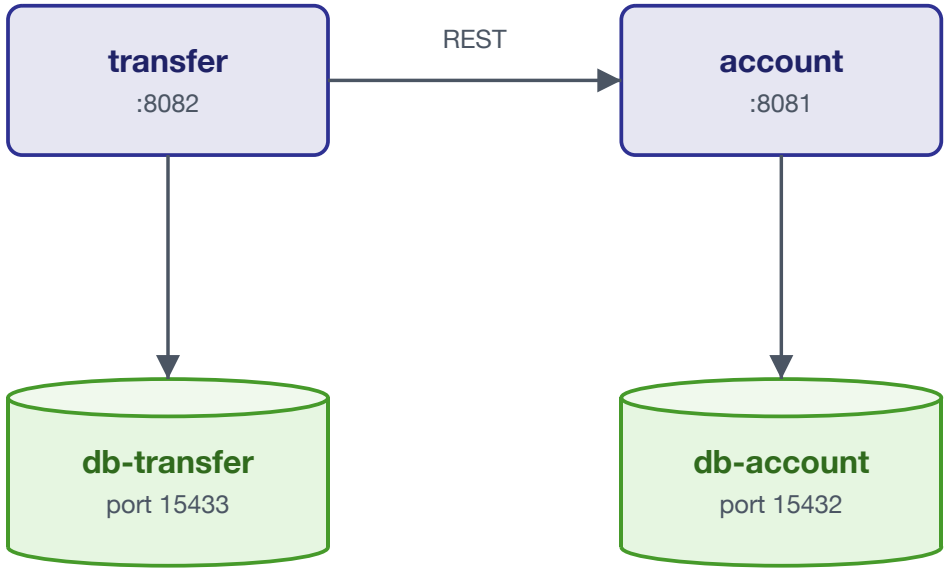
Server bersama dengan database dan kredensial terpisah per service juga memenuhi prinsip ini, asalkan service tidak dapat membaca data milik service lain.

Konsekuensi dan *trade-off*

- *Join* lintas service tidak tersedia, sehingga data milik service lain diambil melalui API
- Debit dan credit tidak dapat dibungkus satu transaksi database (saga, Hari 2–3)
- Jumlah database yang harus di-*backup* dan dipantau bertambah

STUDI KASUS

Database per service di studi kasus



- Setiap service memiliki *container* PostgreSQL sendiri di `compose.yaml`.
- Transfer service tidak memiliki koneksi ke `db-account`, sehingga data rekening hanya dapat diperoleh melalui API account.
- Port PostgreSQL dimulai dari 15432 agar tidak bentrok dengan PostgreSQL lokal di port 5432.

KONSEKUENSI

Debit dan credit tidak dapat dibungkus satu transaksi database

```
-- Monolith: satu database, satu transaksi
begin;
update account set balance = balance - 100000
 where number = '1000000001';
update account set balance = balance + 100000
 where number = '1000000002';
commit;
```

```
# Microservices: dua panggilan jaringan
transfer -> account  POST /accounts/1000000001/debit
transfer -> account  POST /accounts/1000000002/credit
                     (gagal? debit sudah terjadi)
```

Di monolith, debit dan credit cukup dibungkus satu transaksi: bila salah satu gagal, keduanya dibatalkan.

Di microservices, debit dan credit menjadi dua panggilan jaringan ke account service. Bila credit gagal setelah debit berhasil, tidak ada *rollback* otomatis.

- Hari 1: transfer dicatat `FAILED` dan masuk *log*
- Hari 2: desain saga dengan kompensasi dan *idempotency key*
- Hari 3: implementasi saga di Lab 5

03

Ekosistem Go dibandingkan Java, Node.js, dan PHP

Ketersediaan *library* per *concern*, karakter *runtime*, dan alasan memilih Go

EKOSISTEM

Library per *concern* di empat ekosistem

Concern	Go	Java	Node.js	PHP
HTTP dan REST	`net/http` (stdlib), chi, gin	Spring Boot (Spring MVC, WebFlux)	Express, Fastify, NestJS	Laravel, Symfony
gRPC	grpc-go	grpc-java	`@grpc/grpc-js`	*Client*: ekstensi `grpc`; *server*: RoadRunner
Kafka	kafka-go, franz-go, confluent-kafka-go	Kafka *client* dari Apache, Spring Kafka	`@confluentinc/kafka-javascript`	php-rdkafka (ekstensi)
Resiliency	failsafe-go, sony/gobreaker	Resilience4j	opossum	Ganesha (*circuit breaker*)
OAuth2 dan JWT	golang-jwt, `x/oauth2`	Spring Security (OAuth2 *resource server*)	jose, openid-client	firebase/php-jwt, lcobucci/jwt
OpenTelemetry	OTel Go SDK	OTel Java agent, Micrometer	OTel JS SDK	OTel PHP
Akses database	pgx, `database/sql`	JDBC, Spring Data JPA (Hibernate)	node-postgres (`pg`), Prisma, TypeORM	PDO, Doctrine, Eloquent

CATATAN

Keempat ekosistem memiliki *library* untuk setiap *concern*. Perbedaannya ada pada bentuk paket: Java menyediakan integrasi siap pakai melalui Spring, sedangkan Go merangkai *library* kecil per *concern*. Tabel ini hanya memuat *library* yang masih aktif dipelihara (diperiksa Oktober 2026).

EKOSISTEM

Karakter *runtime* yang berpengaruh pada microservices

Aspek	Go	Java	Node.js	PHP
Artefak *deployment*	Satu *binary* statis; *image* dapat berbasis `scratch`	JVM dan file `jar`	*Runtime* Node dan direktori `node_modules`	PHP-FPM dan *web server*, atau *worker* persisten (RoadRunner, Swoole)
Model konkurensi	*Goroutine* yang dijadwalkan *runtime* Go	*Thread* JVM; *virtual thread* sejak Java 21	Satu *event loop*; komputasi berat menahan seluruh *request*	Setiap *worker* FPM melayani satu *request* pada satu waktu; *state* tidak bertahan antar-*request*
Startup dan memori	*Startup* cepat, memori kecil	*Startup* lebih lama karena JVM dan inisialisasi *framework*; dapat dipersingkat dengan GraalVM *native image*	*Startup* cepat	*Bootstrap framework* di setiap *request*, kecuali dengan *worker* persisten

CATATAN

Angka dari studi kasus: *image* account service sekitar 8 MB terkompresi dan *startup* di bawah 1 detik (diukur di Lab 9). Dalam praktik, pilihan bahasa paling sering ditentukan oleh bahasa yang sudah dikuasai tim dan sanggup dirawat tim selama bertahun-tahun.

KEPUTUSAN DESAIN

Go untuk service di studi kasus

Kebutuhan

Banyak service kecil yang berjalan di *container* dan Kubernetes: *image* kecil, *startup* cepat, berjalan sebagai *non-root*, dan mudah dibaca peserta dari berbagai latar belakang bahasa.

Opsi yang dipertimbangkan

Java dengan Spring Boot dan Spring Cloud, Node.js, PHP, dan Go.

Pilihan di studi kasus

Go dengan *standard library* dan *library* kecil per *concern* (lihat tabel *library*). Service pendukung (*registry*, SSO, *log server*, *monitoring*) dan analisis *build*, *reuse*, *buy* dibahas di Hari 4.

Konsekuensi dan *trade-off*

- *Binary* statis, *image* `scratch`, dan konkurensi dengan *goroutine*
- Go tidak memiliki *framework* tunggal setara Spring Boot, sehingga konfigurasi, *middleware*, dan telemetri dirangkai sendiri dengan kode yang ringkas
- Tim yang terbiasa dengan Java atau PHP perlu waktu belajar

04

Go Essentials untuk Service

Fitur bahasa Go yang cukup untuk menulis dan membaca kode service di lab

Module dan *workspace*

```
// studi-kasus/go.work
go 1.26.0

use (
    ./account
    ./transfer
)
```

```
// studi-kasus/account/go.mod
module github.com/artivisi/training-microservices-2026-01/studi-kasus/account

go 1.26.0

require (
    github.com/jackc/pgx-shopspring-decimal v0.0.0-20220624020537-1d36b5a1853e
    github.com/jackc/pgx/v5 v5.11.0
    github.com/shopspring/decimal v1.6.0
)
```

- Setiap *module* didefinisikan oleh `go.mod` dan memiliki versi serta dependensi sendiri.
- Setiap service menjadi *module* sendiri, sehingga dapat di-*build* dan dirilis terpisah.
- `go.work` menggabungkan beberapa *module* lokal agar `go build` dan editor mengenali semuanya sekaligus.

Struct dan JSON

```
type Account struct {
    Number    string          `json:"number"`
    OwnerName string          `json:"owner_name"`
    Balance   decimal.Decimal `json:"balance"`
    CreatedAt time.Time       `json:"created_at"`
}

type CreateRequest struct {
    Number         string          `json:"number"`
    OwnerName      string          `json:"owner_name"`
    InitialBalance decimal.Decimal `json:"initial_balance"`
}
```

- *Struct tag* `json:"..."` menentukan nama *field* di JSON.
- Nominal uang menggunakan `decimal.Decimal`, disimpan sebagai `numeric(19,2)`.
- `float64` tidak dapat menyimpan 0.1 secara tepat, sehingga selisih pembulatan menumpuk pada saldo.
- Kolom `numeric` dipetakan ke `decimal.Decimal` dengan mendaftarkan `pgxdecimal.Register` di `AfterConnect`.

Interface

```go
// Store is the persistence contract; the HTTP handler
// depends on this, not on PostgreSQL.
type Store interface {
    Create(ctx context.Context, req CreateRequest) (Account, error)
    Get(ctx context.Context, number string) (Account, error)
    List(ctx context.Context) ([]Account, error)
    Mutate(ctx context.Context, number string,
        t MutationType, req MutationRequest) (Account, error)
    Ping(ctx context.Context) error
}

type Handler struct {
    store Store
    log   *slog.Logger
}
```

- *Interface* didefinisikan di sisi kode yang menggunakannya, yaitu *handler*.
- Implementasi PostgreSQL memenuhi *interface* secara implisit, tanpa kata kunci `implements`.
- *Unit test* untuk *handler* cukup menggunakan implementasi *in-memory*.

Error handling

```go
// package transfer
var (
    ErrAccountNotFound   = errors.New("account not found")
    ErrInsufficientFunds = errors.New("insufficient funds")
)

// package accountclient: bungkus dengan konteks
case http.StatusUnprocessableEntity:
    return fmt.Errorf("%s %s: %w", kind, number,
        transfer.ErrInsufficientFunds)

// handler transfer: kenali jenis error, lalu pilih status
t, err := h.svc.Execute(ctx, req)
switch {
case err == nil:
    writeJSON(w, http.StatusCreated, t)
case errors.Is(err, ErrAccountNotFound),
    errors.Is(err, ErrInsufficientFunds):
    writeJSON(w, http.StatusUnprocessableEntity, t)
default:
    writeJSON(w, http.StatusBadGateway, t)
}
```

- *Error* berupa nilai biasa yang dikembalikan fungsi dan diperiksa tepat setelah pemanggilan.
- `%w` membungkus *error* dengan konteks tambahan tanpa menghilangkan identitas *error* asalnya.
- `errors.Is` memeriksa seluruh *error chain* yang dibungkus, sehingga *handler* transfer tetap mengenali `ErrInsufficientFunds`.

Goroutine dan *channel*

```
errCh := make(chan error, 1)
go func() {
    log.Info("account service listening", "addr", cfg.HTTPAddr)
    errCh <- srv.ListenAndServe()
}()

select {
case err := <-errCh:
    return err          // server gagal start, misalnya port terpakai
case <-ctx.Done():
    log.Info("shutting down")
    // lanjut ke graceful shutdown
}
```

- `go func()` menjalankan fungsi secara konkuren di *goroutine* baru.
- *Channel* `errCh` mengirim hasil dari *goroutine* tersebut ke fungsi `run`.
- `select` berhenti menunggu begitu salah satu dari dua hal berikut terjadi: server berhenti karena *error*, atau sistem operasi mengirim sinyal berhenti.

`context.Context`

```go
// dibatalkan saat Ctrl+C atau SIGTERM
ctx, stop := signal.NotifyContext(context.Background(),
    os.Interrupt, syscall.SIGTERM)
defer stop()

// batas waktu 2 detik untuk satu operasi
ctx, cancel := context.WithTimeout(r.Context(), 2*time.Second)
defer cancel()
if err := h.store.Ping(ctx); err != nil {
    // database tidak menjawab dalam 2 detik
}
```

- `context` membawa *deadline* dan sinyal pembatalan ke seluruh rantai pemanggilan.
- `signal.NotifyContext` membatalkan `ctx` ketika menerima Ctrl+C atau `SIGTERM` dari Docker dan Kubernetes.
- `WithTimeout` membatasi waktu satu operasi, misalnya *ping* ke database.
- Setiap fungsi yang mengakses jaringan atau database menerima `ctx` sebagai parameter pertama.

`net/http`: *handler* dan *middleware*

```go
type Handler interface {
    ServeHTTP(http.ResponseWriter, *http.Request)
}

// middleware: wraps a handler and returns a new one
func logging(log *slog.Logger, next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        start := time.Now()
        next.ServeHTTP(w, r)
        log.Info("request", "method", r.Method,
            "path", r.URL.Path, "took", time.Since(start))
    })
}
```

Konsep yang sama di ekosistem lain: *middleware* Koa (`await next()`) dan Laravel (`$next($request)`) juga membungkus *handler*, sehingga kode setelah `next` dipanggil berjalan setelah *response* siap. *Middleware* Express berbentuk *chain* `next()`, sehingga kode sesudah *response* membutuhkan `res.on('finish')`. Istilah *interceptor* berbeda arti: *hook* di Spring, *wrapper* di NestJS, dan padanan *middleware* di gRPC.

- Setiap tipe yang memiliki *method* `ServeHTTP` dapat menangani *request*.
- `http.HandlerFunc` mengubah fungsi biasa menjadi *handler*.
- *Middleware* berupa fungsi yang menerima *handler* dan mengembalikan *handler* baru. Pola ini digunakan kembali untuk *fault injection* (Hari 2), JWT (Hari 3), dan metrik (Lab 8).
- Padanan di Spring adalah Servlet `Filter`, yang membungkus seluruh *request*. `HandlerInterceptor` Spring berbeda, karena baru berjalan setelah *controller* dipilih. Padanan *middleware* untuk gRPC disebut *interceptor* (Hari 2).

05

Anatomi Service Go

Struktur *project*, *routing*, validasi, *12-factor*, konfigurasi, dan *health check*

ANATOMI SERVICE

Struktur *project*

```
studi-kasus/
├── go.work
├── compose.yaml        # PostgreSQL per service
├── Makefile            # infra-up, infra-down, test
├── account/
│   ├── go.mod
│   ├── main.go         # run(): rangkai komponen, server
│   ├── config.go       # LoadConfig dari environment
│   ├── Makefile        # run, test, build
│   └── internal/
│       ├── account/    # model, validasi, store, HTTP
│       └── fault/      # fault injection (Hari 2)
└── transfer/
    ├── main.go, config.go, Makefile
    └── internal/
        ├── transfer/       # model, service, store, HTTP
        └── accountclient/  # client REST ke account
```

- Kode dikelompokkan per fitur (*package-by-feature*) di bawah `internal/`.
- *Package* di `internal/` tidak dapat di-*import* oleh *module* lain.
- `main.go` hanya merangkai komponen, sedangkan logika bisnis berada di *package* domain.
- Setiap service memiliki `Makefile` sendiri. Target yang sama nantinya dipanggil *pipeline* CI.

Routing dengan `ServeMux`

```go
func (h *Handler) Routes() *http.ServeMux {
    mux := http.NewServeMux()
    mux.HandleFunc("POST /accounts", h.create)
    mux.HandleFunc("GET /accounts", h.list)
    mux.HandleFunc("GET /accounts/{number}", h.get)
    mux.HandleFunc("POST /accounts/{number}/debit", h.mutate(Debit))
    mux.HandleFunc("POST /accounts/{number}/credit", h.mutate(Credit))
    mux.HandleFunc("GET /healthz", h.healthz)
    mux.HandleFunc("GET /readyz", h.readyz)
    return mux
}

func (h *Handler) get(w http.ResponseWriter, r *http.Request) {
    a, err := h.store.Get(r.Context(), r.PathValue("number"))
    // ...
}
```

- Sejak Go 1.22, pola *route* memuat *method* dan *path* sekaligus.
- `{number}` dibaca dengan `r.PathValue("number")`.
- *Request* dengan *method* yang tidak terdaftar otomatis mendapat *response* `405 Method Not Allowed`.
- Sejak Go 1.23, `r.Pattern` berisi pola yang cocok, misalnya `GET /accounts/{number}`. Lab 8 menggunakannya sebagai label metrik, sehingga nomor rekening tidak masuk ke label.
- Sejak Go 1.26, *redirect* untuk garis miring di akhir *path* menggunakan `307`, sehingga *method* `POST` tetap `POST`.
- Kebutuhan *routing* di lab terpenuhi tanpa *router* pihak ketiga.

PERBANDINGAN

`ServeMux`, chi, gin, echo, dan Fiber

Aspek	`http.ServeMux`	chi	gin	echo	Fiber
HTTP engine	`net/http`	`net/http`	`net/http`	`net/http`	fasthttp
Bentuk *handler*	`http.Handler`	`http.Handler`	`func(*gin.Context)`	`func(*echo.Context) error`	`func(fiber.Ctx) error`
Middleware `net/http`	Langsung digunakan	Langsung digunakan	Lewat *wrapper*	Lewat *wrapper*	Lewat *adaptor*, dengan *overhead*
Grup *route* dengan *middleware* per grup	Tidak ada, dirangkai sendiri	Ada	Ada	Ada	Ada
Binding dan validasi	Ditulis sendiri	Ditulis sendiri	Bawaan (go-playground/validator)	*Binding* bawaan, *validator* dipasang sendiri	*Binding* bawaan, *validator* dipasang sendiri
HTTP/2	Ya	Ya	Ya	Ya	Belum didukung fasthttp
Dependensi langsung	Tidak ada	Tidak ada	15	3	15
Versi dan lisensi (Okt 2026)	Bagian dari Go	v5.5, MIT	v1.12, MIT	v5.4, MIT	v3.6, MIT

Pindah antar-*router*

Handler `ServeMux` dan chi sama-sama `http.Handler`, sehingga pindah ke chi tidak mengubah *handler*. Pindah ke gin, echo, atau Fiber berarti menulis ulang setiap *handler*.

Catatan Fiber

Nilai dari `fiber.Ctx` digunakan ulang setelah *handler* selesai, sehingga tidak boleh disimpan. README fasthttp sendiri menyarankan `net/http` untuk sebagian besar kasus.

Gunakan *framework* bila

API memiliki banyak grup *route* dengan *middleware* berbeda per grup, membutuhkan *binding* dan validasi deklaratif, atau tim sudah terbiasa dengan *framework* tersebut.

KEPUTUSAN DESAIN

Routing dengan `ServeMux` dari *standard library*

Kebutuhan

Routing REST berdasarkan *method* dan *path*, dengan parameter di *path* (`/accounts/{number}`), serta *middleware* untuk *logging*, JWT, dan metrik.

Opsi yang dipertimbangkan

- `http.ServeMux`: pola *method* dan *path* sejak Go 1.22
- chi: *router* yang kompatibel dengan `http.Handler`
- gin, echo, Fiber: *framework* dengan *binding* dan validasi (Fiber di atas fasthttp, bukan `net/http`)

Pilihan di studi kasus

`http.ServeMux` tanpa *router* pihak ketiga. *Middleware* ditulis sebagai fungsi yang menerima dan mengembalikan `http.Handler`.

Konsekuensi dan *trade-off*

- Tidak ada dependensi tambahan. *Handler* tetap berupa `http.Handler` biasa, sehingga dapat dipasang di chi bila kelak dibutuhkan
- Tidak tersedia pengelompokan *route* dengan *middleware* per grup, maupun *binding* dan validasi otomatis seperti di gin
- Validasi ditulis sendiri (slide “Validasi input”)

Validasi input

```go
type ValidationError map[string]string

func (r CreateRequest) Validate() error {
    v := ValidationError{}
    if !numberPattern.MatchString(r.Number) {
        v["number"] = "must be exactly 10 digits"
    }
    if strings.TrimSpace(r.OwnerName) == "" {
        v["owner_name"] = "must not be empty"
    }
    if r.InitialBalance.IsNegative() {
        v["initial_balance"] = "must not be negative"
    }
    if len(v) > 0 {
        return v
    }
    return nil
}
```

```
HTTP 400
{"error": "validation failed",
 "fields": {"number": "must be exactly 10 digits"}}
```

- Validasi dijalankan sebelum data disimpan ke database.
- `ValidationError` mengumpulkan semua *field* yang bermasalah, sehingga klien menerima seluruh daftar kesalahan dalam satu *response*.
- JSON dengan *field* yang tidak dikenal ditolak (`DisallowUnknownFields`).

ANATOMI SERVICE

12-factor app

Dua belas prinsip untuk aplikasi yang dijalankan di *cloud* (12factor.net). Prinsip ini menjadi acuan sepanjang training.

Faktor	Nama	Penerapan di studi kasus	Dibahas di
I	*Codebase*	Satu *module* dan satu *pipeline* per service; tidak dibahas khusus	—
II	*Dependencies*	Dependensi dideklarasikan di `go.mod` dan `go.sum`	Lab 1
III	*Config*	Konfigurasi hanya dari *environment variable*	Lab 1, 9
IV	*Backing services*	Database, account service, Kafka, Hydra, dan *collector* diakses melalui alamat dari konfigurasi	Lab 1–2, 5, 7, 8
V	*Build, release, run*	*Image* dibangun sekali, lalu dirilis dengan konfigurasi per *environment*	Lab 9–10
VI	*Processes*	Service tidak menyimpan *state* di memori; semua data ada di database	Lab 1, 9
VII	*Port binding*	Service membuka port HTTP dan gRPC sendiri (`HTTP_ADDR`, `GRPC_ADDR`)	Lab 1, 4
VIII	*Concurrency*	Kapasitas ditambah dengan memperbanyak replika	Lab 9
IX	*Disposability*	*Startup* cepat dan *graceful shutdown*	Lab 1, 9–10
X	*Dev/prod parity*	*Image* dan *manifest base* yang sama di k3d dan di *server* pelatihan	Lab 9–10
XI	*Logs*	*Log* ditulis ke `stdout` sebagai aliran *event*; format JSON sejak Lab 8	Lab 1, 8
XII	*Admin processes*	Skema database diterapkan saat *startup* service, tanpa proses admin terpisah; tidak dibahas khusus	—

Referensi: Adam Wiggins, *The Twelve-Factor App*, Heroku, `12factor.net` (sumbernya terbuka di `github.com/twelve-factor/twelve-factor`). Kevin Hoffman, *Beyond the Twelve-Factor App*, O'Reilly, 2016, menambah tiga faktor: *API first*, *telemetry*, serta *authentication* dan *authorization*.

Konfigurasi dari *environment variable*

```go
func LoadConfig() (Config, error) {
    var missing []string
    get := func(name string) string {
        v := os.Getenv(name)
        if v == "" {
            missing = append(missing, name)
        }
        return v
    }
    cfg := Config{
        HTTPAddr:    get("HTTP_ADDR"),
        DatabaseURL: get("DATABASE_URL"),
    }
    if len(missing) > 0 {
        return Config{}, fmt.Errorf(
            "missing required environment variables: %s",
            strings.Join(missing, ", "))
    }
    return cfg, nil
}
```

```
level=ERROR msg="account service stopped"
  error="missing required environment variables: DATABASE_URL"
```

- Konfigurasi hanya dibaca dari *environment variable* (faktor III, *Config*), sehingga *image* yang sama berjalan di laptop, CI, dan Kubernetes.
- Semua variabel wajib diisi dan tidak ada nilai *default*.
- Bila ada variabel yang kosong, service berhenti saat *startup* dengan *error* yang menyebut nama variabel tersebut.
- Di proyek sungguhan, *library* seperti caarlos0/env membaca *environment variable* langsung ke *struct* (`env:"DATABASE_URL,required"`), termasuk konversi tipe. Viper dan koanf menambahkan *file* konfigurasi (YAML/TOML) dan *flag*, dan umum digunakan di aplikasi CLI.

Liveness dan *readiness* menjawab pertanyaan yang berbeda

Liveness: `GET /healthz`

Pertanyaan: apakah proses ini macet dan perlu di-*restart*?

Yang bertindak: kubelet di *node* tempat *pod* berjalan.

Tindakan bila gagal: *container* di-*restart*.

Yang diperiksa di studi kasus: tidak ada dependensi. *Endpoint* menjawab `200` selama server HTTP masih melayani *request*.

Readiness: `GET /readyz`

Pertanyaan: apakah *pod* ini boleh menerima *traffic* sekarang?

Yang bertindak: kubelet melaporkan hasilnya, lalu *pod* dikeluarkan dari daftar *endpoint* Service.

Tindakan bila gagal: *pod* tidak menerima *request* baru, tetapi *container* tidak di-*restart*. Begitu kembali `200`, *pod* menerima *traffic* lagi.

Yang diperiksa di studi kasus: *ping* ke database milik service sendiri dengan batas 2 detik, `503` bila gagal.

startupProbe menunda *liveness* dan *readiness* untuk service yang *startup*-nya lama. Studi kasus tidak memerlukannya. YAML ketiga *probe* dibahas di Lab 9.

Skenario *liveness* dan *readiness*

Skenario	`/healthz`	`/readyz`	Tindakan Kubernetes	Alasan
Startup, service sedang menghubungi database	Belum menjawab	Belum menjawab	*Pod* belum menerima *traffic*. Bila koneksi gagal, proses berhenti dengan *error* dan *container* di-*restart*	Server HTTP baru dibuka setelah *ping* ke database berhasil. Bila tahap ini lebih lama dari batas *liveness*, gunakan *startupProbe*
Database mati sementara	`200`	`503`	*Pod* dikeluarkan dari Service, tanpa *restart*	*Restart* tidak memperbaiki database. Bila *liveness* ikut memeriksa database, semua *pod* di-*restart* bersamaan
Proses macet, server HTTP tidak menjawab	*Timeout*	*Timeout*	*Container* di-*restart*	Hanya *restart* yang mengembalikan proses yang macet
Overload	Lambat	Lambat atau `503`	*Pod* dikeluarkan sementara dari Service	Beban beralih ke replika lain. Batas *liveness* dibuat longgar agar beban sesaat tidak memicu *restart* yang menambah beban
Account service lambat, dilihat dari transfer service	`200`	`200`	Tidak ada	*Readiness* transfer hanya memeriksa database transfer. Gangguan account ditangani *timeout* dan *circuit breaker* (Hari 2)

PERHATIAN

Mengapa perlu dua *probe*? Gangguan terbagi dua jenis. Sebagian hilang dengan sendirinya, misalnya database yang sementara mati: *pod* cukup berhenti menerima *traffic* sambil menunggu (*readiness*). Sebagian lagi hanya hilang dengan *restart*, misalnya proses yang macet (*liveness*). Tindakan untuk kedua jenis gangguan ini berlawanan, sehingga masing-masing membutuhkan *probe* sendiri.

Graceful shutdown

```
srv := &http.Server{
    Addr:              cfg.HTTPAddr,
    Handler:           mux,
    ReadHeaderTimeout: 5 * time.Second,
}
// ... server berjalan di goroutine (slide "Goroutine dan channel")

case <-ctx.Done():
    log.Info("shutting down")
    shutdownCtx, cancel := context.WithTimeout(
        context.Background(), 10*time.Second)
    defer cancel()
    if err := srv.Shutdown(shutdownCtx); err != nil &&
        !errors.Is(err, http.ErrServerClosed) {
        return err
    }
    return nil
```

- Saat menerima `SIGTERM`, server berhenti menerima koneksi baru.
- `srv.Shutdown` menunggu *request* yang sedang berjalan selesai, dengan batas 10 detik.
- Batas ini berbeda per service. Sejak Lab 5, transfer menunggu hingga 60 detik agar saga tidak terpotong di antara debit dan credit.
- *Rolling update* di Kubernetes (Hari 4) mengandalkan perilaku ini agar tidak ada transfer yang terputus di tengah proses (faktor IX, *Disposability*).

Pemetaan *error* ke status HTTP

Kondisi	*Error* di Go	Status HTTP
Body berisi JSON yang tidak valid atau memuat *field* tak dikenal	Proses *decode* gagal	`400`
Isi *field* tidak memenuhi aturan	`ValidationError`	`400` beserta daftar *field*
Nomor rekening sudah terdaftar	`ErrAlreadyExists`	`409`
Rekening tidak ditemukan	`ErrNotFound`	`404`
Saldo tidak mencukupi untuk debit	`ErrInsufficientFunds`	`422`
Method tidak terdaftar untuk *path* tersebut	Ditangani `ServeMux`	`405`
Error lain saat melayani *request* API, misalnya database terputus	*Error* apa pun	`500`
`GET /readyz` saat *ping* ke database gagal	*Ping* gagal	`503`

CATATAN

`500` berarti satu *request* gagal karena *error* yang tidak terduga. Klien hanya menerima `"internal error"`, sedangkan detailnya, seperti alamat database atau isi *query*, hanya ditulis ke *log*. `503` dari `/readyz` berarti *instance* ini belum siap menerima *traffic*, sehingga Kubernetes mengeluarkan *pod* tersebut dari Service.

06

Lab 1: Account Service

REST, PostgreSQL, validasi, dan *health check*

LAB 1

Mengembangkan account service dari nol

Tujuan

Service pertama yang berjalan lengkap: menerima *request* REST, memvalidasi input, menyimpan data ke database miliknya sendiri, dan melaporkan kondisinya melalui *health check*.

Rujukan

- Langkah lengkap: **Panduan Lab, bab Lab 1**
- Kondisi akhir: *tag* `lab-01`
- Port: HTTP `8081`, PostgreSQL `15432`

Yang dikerjakan

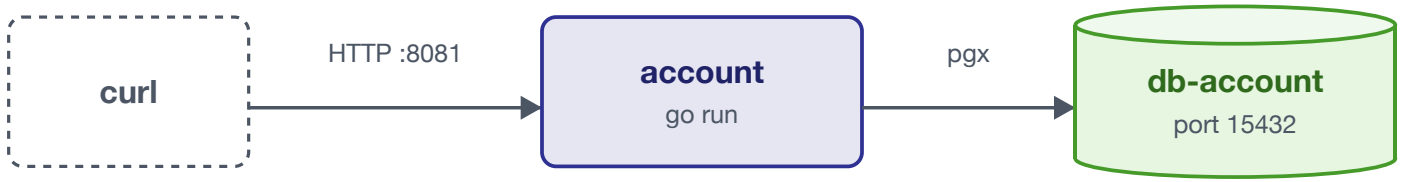
- `POST /accounts`, `GET /accounts`, `GET /accounts/{number}`
- `POST /accounts/{number}/debit` dan `/credit` beserta tabel `mutation`
- `GET /healthz` dan `GET /readyz`
- `LoadConfig` dengan variabel wajib `HTTP_ADDR` dan `DATABASE_URL`

TIPS

Jalankan `make test` sebelum melanjutkan ke Lab 2. *Unit test* untuk *handler* menggunakan `httptest` tanpa database.

LAB 1

Menjalankan dan mencoba account service




```
cd studi-kasus
make infra-up                # PostgreSQL di Docker Compose
make -C account run          # HTTP_ADDR=:8081, DATABASE_URL=...localhost:15432

curl -s -X POST localhost:8081/accounts -H 'Content-Type: application/json' \
  -d '{"number":"1000000001","owner_name":"Budi","initial_balance":"500000.00"}'
curl -s localhost:8081/accounts/1000000001
curl -s localhost:8081/readyz
```

KEPUTUSAN DESAIN

pgx tanpa ORM

Kebutuhan

Akses PostgreSQL dengan SQL yang terlihat jelas, kendali penuh atas transaksi, dan kondisi saldo yang diperiksa di klausa `where` (slide “Mencegah saldo negatif saat dua debit bersamaan”).

Opsi yang dipertimbangkan

- pgx langsung (`pgxpool`)
- `database/sql` dengan *driver* PostgreSQL
- sqlc: kode Go dibangkitkan dari SQL
- ORM: GORM, ent

Pilihan di studi kasus

pgx v5 dengan `pgxpool`. SQL ditulis langsung dengan `pgx.NamedArgs`, dan hasil *query* dipetakan ke *struct* dengan `pgx.RowToStructByPos`.

Konsekuensi dan *trade-off*

- Setiap *query* terbaca apa adanya, termasuk fitur khusus PostgreSQL
- Urutan kolom di `returning` dan `select` harus sama dengan urutan *field* di *struct*
- Perubahan skema diikuti perubahan *query* secara manual, dan kode terikat ke PostgreSQL

LAB 1

Mencegah saldo negatif saat dua debit bersamaan

```
tx, err := s.pool.Begin(ctx)
// ...
defer tx.Rollback(ctx) // no-op after Commit

rows, err := tx.Query(ctx, `
    update account set balance = balance + @delta
    where number = @number and balance + @delta >= 0
    returning number, owner_name, balance, created_at`,
    pgx.NamedArgs{"number": number, "delta": delta})
// tidak ada baris: rekening tidak ada, atau saldo kurang

_, err = tx.Exec(ctx, `
    insert into mutation (account_number, type, amount, reference)
    values (@number, @type, @amount, @reference)`, ...)
return a, tx.Commit(ctx)
```

- Pemeriksaan saldo berada di klausa `where`. Dua debit yang datang bersamaan tidak dapat sama-sama lolos: debit kedua menunggu *row lock*, lalu kondisinya dievaluasi ulang dengan saldo terbaru.
- Perubahan saldo dan baris `mutation` ditulis dalam satu transaksi.
- *Constraint* `check (balance >= 0)` menjadi pengaman terakhir di database.

07

Lab 2: Transfer Service

Dua service, dua database, dan komunikasi REST pertama

LAB 2

Mengembangkan transfer service sebagai service kedua

Tujuan

Mengalami langsung komunikasi antarservice: transfer service menyimpan transfer di database sendiri, lalu meminta account service melakukan debit dan credit melalui REST.

Yang dikerjakan

- `POST /transfers` dan `GET /transfers/{id}`
- Tabel `transfer` dengan status `PENDING`, `COMPLETED`, `FAILED`
- *Package* `accountclient`: client REST ke account service
- Variabel wajib baru: `ACCOUNT_SERVICE_URL` (faktor IV: account service diakses melalui alamat dari konfigurasi)

Rujukan

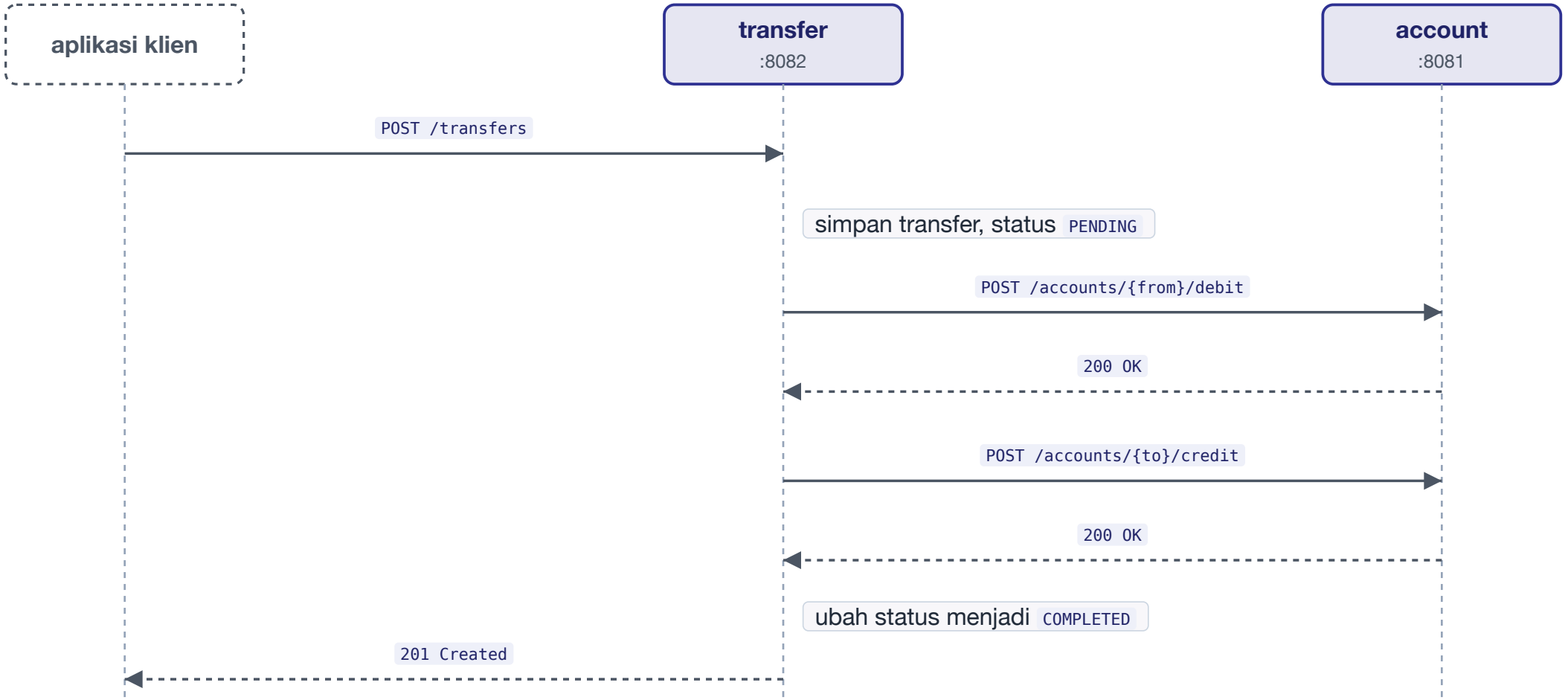
- Langkah lengkap: **Panduan Lab, bab Lab 2**
- Kondisi akhir: *tag* `lab-02`
- Port: HTTP `8082`, PostgreSQL `15433`

CATATAN

Account service dari Lab 1 harus tetap berjalan di port `8081` selama Lab 2.

LAB 2

Alur transfer yang berhasil



LAB 2

Status transfer dan *response* HTTP

```go
func (s *Service) Execute(ctx context.Context,
    req CreateRequest) (Transfer, error) {
    t, err := s.store.Create(ctx, req) // PENDING
    if err != nil {
        return Transfer{}, fmt.Errorf("record transfer: %w", err)
    }
    if err := s.accounts.Debit(ctx, t.FromAccount,
        t.Amount, t.ID); err != nil {
        return s.fail(ctx, t, "debit failed", err)
    }
    if err := s.accounts.Credit(ctx, t.ToAccount,
        t.Amount, t.ID); err != nil {
        return s.fail(ctx, t, "credit failed after debit", err)
    }
    return s.store.Finish(ctx, t.ID, Completed, "")
}
```

Hasil	HTTP
Transfer selesai	`201`
Rekening tidak ada atau saldo kurang	`422`
Account service gagal atau tidak dapat dihubungi	`502`

Transfer yang gagal tetap tersimpan dengan status `FAILED` dan `failure_reason`, sehingga dapat ditelusuri melalui `GET /transfers/{id}`.

LAB 2

Celah yang sengaja dibiarkan di Lab 2

Client tanpa *timeout*

`accountclient` menggunakan `http.Client` tanpa *timeout*. Bila account service lambat, setiap transfer ikut menunggu tanpa batas, dan *goroutine* serta koneksi menumpuk di transfer service.

Ditangani di Lab 3 dengan *timeout*, *retry*, dan *circuit breaker*.

Credit gagal setelah debit

Dana sudah keluar dari rekening sumber, tetapi tidak sampai di rekening tujuan. Transfer dicatat `FAILED` dan *log* mencatat bahwa *reversal* manual diperlukan.

Ditangani di Lab 5 dengan saga: debit yang sudah terjadi dibatalkan melalui transaksi kompensasi.

TIPS

Lab 3 diawali dengan *fault injection* pada account service (`PUT /admin/fault`) untuk memunculkan *delay* dan *error* secara sengaja, sehingga kedua celah ini terlihat langsung.

PENUTUP HARI 1

Ringkasan dan persiapan Hari 2

Hasil Hari 1 pada peta *concern*

- **#1 *Decomposition*:** *service boundary* dari kapabilitas bisnis, satu database per service
- **#2 Komunikasi:** transfer memanggil account melalui REST
- **#7 *Configuration*:** variabel wajib dari *environment variable*, tanpa nilai *default* (faktor III)
- **#9 *Deployment* (konsep):** *liveness* dan *readiness* dipisahkan sejak awal, *graceful shutdown* (faktor IX)
- Dua celah dicatat untuk #3 *Resiliency* dan #4 Konsistensi data
- Keputusan desain: Go, `ServeMux`, pgx tanpa ORM, *database per service*

Persiapan Hari 2

- Pastikan `make test` lulus dan kode sesuai *tag* `lab-02`
- Pastikan `protoc`, `protoc-gen-go`, dan `protoc-gen-go-grpc` terpasang (Lab 4)
- *Concern* Hari 2: #3 *Resiliency* (*timeout*, *retry*, *circuit breaker*), #2 gRPC untuk panggilan internal, dan #4 konsistensi data lintas service